Technical Sheet Loopy

Nicola Avellino, Simone Battisti, Liam Demattè, Riccardo Gonzato

Contents

1 Overview of the System **2**

2 System Architecture **2**

3 Components Details **2**
3.1 Application Server 1 . . . 2
3.2 Application Server 2 . . . 2
3.3 Mobile Application . . . 3
3.4 Glucose Calculator . . . 3
3.5 Metric Calculator . . . 3
3.6 Raspberry Pi Server . . . 4

4 Raspberry Pi Specifics and Components **4**

5 Loopy AI **4**

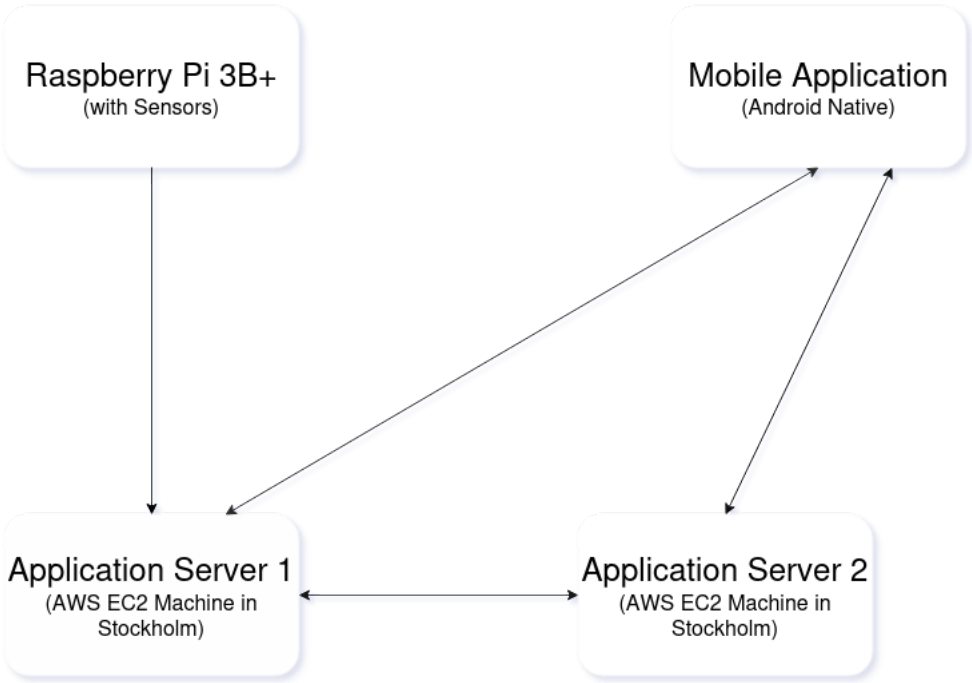
6 Server Informations **4**
6.1 Application Server 1 . . . 4
6.2 Application Server 2 . . . 5
6.3 Raspberry Pi Server . . . 5

7 Database Tables Informations **5**
7.1 AWS EC2 Machine 1 Tables and Raspberry Pi Tables . . . 5
7.2 AWS EC2 Machine 2 Tables . . . 6

1 Overview of the System

Loopy is a health monitor system that uses a wearable device arm bend to scan health values from the user. The system is composed by multiple distributed systems that works in parallel to retrieve, elaborate and give health data to the user in base of his daily activities.

2 System Architecture



3 Components Details

3.1 Application Server 1

Application Server 1 is a Kotlin Ktor web server that manages all the most basic input outputs of the system. It manages:

- Login/Register Logic
- Data Storing Logic
- Device Status Logic

Application Server 1 runs on a AWS EC2 machine in Stockholm, in particular a t2.nano machine with 30 gb of storage and 1 gb of RAM.
It's used from the whole devices in the system.

3.2 Application Server 2

Application Server 2 is another Kotlin Ktor web server that manages all the calculations on data retrived by the raspberry and saved on Application Server 1. It manges:

- Machine Learning Logic for Glucose Calculation.
- AI Chatbot Loopy Logic (Kotlin Koog)
- Graphs Management (Kotlin Kandy)

Application Server 2 to work uses the data from Application Server 1 with with a specific endpoint. Application Server 2 is mainly used from the Mobile App because.

3.3 Mobile Application

The mobile application is an Android SDK Native application that permits the full overview on the system and the data retrieved from the device. The application is divided into 5 main pages:

- **MainActivity Page**: the home page of the application with all the main functionalities Loopy can offer, like the *Graph Generation, Loopy ChatBot Overview and summary of daily heart rate and consumption.*
- **DataActivity Page**: data activity shows the user the whole data got from the Raspberry Pi during the day.
- **ChatActivity Page**: chatbot page where you can iteract with the relative endpoint and input question to the Koog AI chatbot.
- **ProfileActivity Page**: shows the profile informations.
- **SettingsActivity Page**: shows the various miscellaneous of configuration of the app.
- **Login/Register Page**: permits the user to register or login to the application.

3.4 Glucose Calculator

Glucose calculator is a python tool that permits to avaluate a value of the glycemic index of the user in base of the data retrieved during the day. To make this there is a Machine Learning algorithm trained on sample data CSV from https://physionet.org/content/wearable-device-dataset/1.0.1/ defined as data retrived from a smartwatch.
The algorithm specifically is a **Random Forest Algorithm** that saves a model.pkl into the AWS Machine and call it when the user requests a glucose prediction. At the same time we use **KNN** to evaluate some NaN values that are stored in the CSVs and we also add to them some attributes like:

- **hr_mean_5min**: defines a mean of the HR of the last 5 minutes
- **eda_mean_5min**: defines a mean of the sweating of the last 5 minutes
- **temp_mean_15min**: defines a mean of the temperature of the last 5 minutes
- **hr_diff_1min**: value of the HR in a delta of 1 minute
- **temp_diff_1min**: value of the temperature in a delta of 1 minute
- **hr_std_15min**: standard deviation of the HR in the last 15 minutes

All of these data will help the training of the model to be more precise and fast.
The result of the operation is saved on the Database and retrieved by the specific endpoint

3.5 Metric Calculator

Metric calculator is a python project specified on the calculation of **Daily Stress**, **Daily Activity** and **Last 2 Days Sleep**. It's divided into 2 main python files:

- **daily_metrics_calc.py**: this file contains (as the name says) the whole logic of calculation of the values during the day. These are Stress and Activity. Activity is calculated using the accelerometer data on the Raspberry Pi, Stress instead is calculated using the sweating and HR data during the day. With an algorithm can define some *bands* of Stress or Activity. Stress Level can be:
  - **Calm**
  - **Medium**
  - **High**

  Activity level can be:
  - **Sedentary**
  - **Light**
  - **Moderate**
  - **High**
- **night_metrics_calc.py**: this file contains the whole logic for calculating the Sleep values during the night and also the Stress ones. The sleep is calculated using HR, sweating and accelerometer data during the night. Also the Sleep algorithm returns levels of it:
  - **Awake**
  - **Light**
  - **REM**
  - **Deep**

3.6 Raspberry Pi Server

Raspberry Pi server logic defines, as the name says, the full server and sensors logic installed on the Raspberry Pi. As sad before on the Raspberry Pi runs a Kotlin Ktor Server that is connected directly with the AWS Machine 1 (and so Application Server 1) to share the data retrieved from the sensors. Sensors logic is made using C++.

4 Raspberry Pi Specifics and Components

The Raspberry Pi specifically is a **Raspberry Pi 3B+** with 4 GB of RAM and a 32 GB microsd. On it is mounted a **Raspberry Pi Hat** to connect all the sensors. The sensors used by Loopy Device are:

- **Accelerometer/Gyroscope** → GY - BNO055
- **PPG** → MAX30102
- **Electrodes** → TMP102
- **Thermometer** → GY - BNO055

For the use of the Electrodes on the Hat is mounted an ADC that permits the reading of analogic data.

5 Loopy AI

As sad before Loopy uses a LLM to create a chatbot where the user can chat and know various informations about his data and advices on his lifestyle. Specifically is used **Kotlin Koog AI** a new library deveolped by JetBrains team that can define agents management in Kotlin. We defined an AI agent called Loopy without a saved memory, using as LLM "*Google Gemini 2.5 Flash*" from Google AI Studio.
The logic behind this is the calculation from the DB of all the means of the data retrieved during the day, we pass them into the Koog Agent Prompt, having a response time of 4/5 seconds. Loopy AI doesn't store messages or personal informations.

6 Server Informations

Loopy defines 3 main servers: **Appication Server 1**, **Application Server 2** and **Raspberry Pi Server**. All the server are Kotlin Ktor server, specifically the Maven Version 3.2.1. The Jsons are made using Kotlin Serialization plugin version 2.2.20. For each server we have different endpoints, here the full list:

6.1 Application Server 1

- [**POST**] `/login`: gets a LoginJson that contains username/email and password of the account and checks if the account is valid for the login or not.
- [**POST**] `/register`: gets a RegisterJson that contains username, password, email, height, weight, gender and age of the user and registers the user into the loopy platform.
- [**POST**] `/user/{id}`: gets a parameter into the path that defines the user id and returns all the informations about the user.
- [**POST**] `/editUser/{id}`: gets the user id as parameter and also a RegisterJson to modify all the account informations.
- [**POST**] `/saveData/{id}`: receives a SaveDataJson that contains all the data retrieved from the Raspberry. This endpoint is called only from the Raspberry to the AWS machine to save the data.
- [**GET**] `/data/csv/{id}`: receives the id as parameter and returns all the data of HR, sweating and thermometer in a CSV style. This is used from the ML to retrieve all the data it needs to provide a glucose prediction.
- [**GET**] `/getData/{id}`: receives the id as parameter and returns a ReturnDataJson that contains the mean of Temperature, HR, Blood Oxygen and Sweating. Is used from the AI logic and the Mobile Application.
- [**GET**] `/status/{id}`: receives the id as parameter and returns a StatusJson that gives the Mobile Application the status of all the sensors on the Raspberry.
- [**POST**] `/saveStatus/{id}`: receives the id as parameter and is called from the Raspberry Pi to save the current status on the AWS Machine DB. This endpoint is called each 5 seconds from the Raspberry with a crontab.

6.2 Application Server 2

- [**POST**] `/agentProcess/{id}`: reveices an AgentJson that contains the username and the query the user wants to input to Loopy AI ChatBot. It returns a string with the response of the LLM
- [**GET**] `/trainModel`: trains the model in base of the most recent CSV data saved on the machine.
- [**GET**] `/modelProcess/{id}`: receives the id as parameter and returns the glucose predict defined by the ML logic. Saves also the data on the machine DB.
- [**GET**] `/getSSAGData/{id}`: receives the id as parameter and returns an ReturnSSAGDataJon with the values of Stress Level, Sleep Level, Activity Level and Glucose Level that are mainly calculated on this machine.
- [**GET**] `/generateGraph/{graphType}/{id}`: revices the id as parameter and a paramter *graphType* that can be "*stress*", "*sleep*" or "*activity*" and defines the type of graph the server returns. It returns a bitmap image of the graph that is rendered into the Mobile Application.

6.3 Raspberry Pi Server

- [**POST**] `/saveData`: receives a SaveDataJson that contains HR, Sweating, Thermometer and Accelerometer and calls the Application Server 1 endpoint `/saveData/{id}` to save the data into the local DB.
- [**POST**] `/status`: executes all sensors and watches for errors, if there are it saves it returns a *FAILURE* if nor it returns *SUCCESS*.
- [**POST**] `/thermometer`: executes the thermometer sensor script that saves the value into the local DB and returns *SUCCESS* or *FAILURE*.
- [**POST**] `/ppg`: executes the PPG sensor script that saves the value into the local DB and returns *SUCCESS* or *FAILURE*.
- [**POST**] `/accelerometer`: executes the accelerometer sensor script that saves the value into the local DB and returns *SUCCESS* or *FAILURE*.
- [**POST**] `/electrodes`: executes the electrodes sensor script that saves the value into the local DB and returns *SUCCESS* or *FAILURE*.
- [**POST**] `/saveStatus`: runs all the sensors and checks for errors, then calls the Application Server 1 `/saveStatus/{id}` that saves on the DB the status of the Raspberry. This endpoint is called every 5 seconds with a crontab.

7 Database Tables Informations

Loopy uses relational databases to save all the informations. To connect the databases to the server uses Kotlin Exposed (Maven Version 0.50.0) that permits the simple connection to the database using table relations as Kotlin classes. To setup the connection to the DB uses Java JDBC connection Maven Version 8.0.33.
As before each server machine contains and manages different tables:

7.1 AWS EC2 Machine 1 Tables and Raspberry Pi Tables

- **Accelerometer**:
  - *id* - Int → PRIMARY KEY AUTO_INCREMENT
  - *acc_x* - Double → acceleration on x axis
  - *acc_y* - Double → acceleration on y axis
  - *acc_z* - Double → acceleration on z axis
  - *userId* - Int → userId foreign key
  - *timestamp* - varchar(100) → contains the timestamp of when the data has imputed.
- **PPG**:
  - *id* - Int → PRIMARY KEY AUTO_INCREMENT
  - *HR* - Double → Heart rate value
  - *oxygen* - Double → bool oxygen value
  - *userId* - Int → userId foreign key

  - *timestamp* - varchar(100) → contains the timestamp of when the data has imputed.

- **Thermometer**:
  - *id* - Int → PRIMARY KEY AUTO_INCREMENT
  - *temperature* - Double → temperature value
  - *userId* - Int → userId foreign key

- *timestamp* - varchar(100) → contains the timestamp of when the data has imputed.

- **Electrodes**:
  - *id* - Int → PRIMARY KEY AUTO_INCREMENT
  - *sweating* - Double → sweating value
  - *userId* - Int → userId foreign key
  - *timestamp* - varchar(100) → contains the timestamp of when the data has imputed.

- **User**:
  - *id* - Int → PRIMARY KEY AUTO_INCREMENT
  - *username* - varchar(100) → user's account username
  - *email* - varchar(100) → user's account email
  - *password* - varchar(100) → user's account password
  - *height* - Int → user's height
  - *weight* - Int → user's weight
  - *age* - Int → user's age
  - *sex* - varchar(100) → user's gender

- **SensorsStatus**:
  - *id* - Int → PRIMARY KEY AUTO_INCREMENT
  - *PPGStatus* - varchar(2) → PPG status
  - *ElectrodeStatus* - varchar(2) → Electrodes status
  - *AccelerometerStatus* - varchar(2) → Accelerometer status
  - *ThermometerStatus* - varchar(2) → Thermometer status
  - *userId* - Int → userId foreign key
  - *timestamp* - varchar(100) → contains the timestamp of when the data has imputed.

7.2 AWS EC2 Machine 2 Tables

- **Activity**:
  - *id* - Int → PRIMARY KEY AUTO_INCREMENT
  - *activity_level* - Int → activity level value in integer
  - *userId* - Int → userId foreign key
  - *timestamp* - varchar(100) → contains the timestamp of when the data has imputed.

- **Stress**:
  - *id* - Int → PRIMARY KEY AUTO_INCREMENT
  - *stress_level* - Int → stress level value in integer
  - *userId* - Int → userId foreign key
  - *timestamp* - varchar(100) → contains the timestamp of when the data has imputed.

- **Sleep**:
  - *id* - Int → PRIMARY KEY AUTO_INCREMENT
  - *sleep_level* - Int → sleep level value in integer
  - *userId* - Int → userId foreign key
  - *timestamp* - varchar(100) → contains the timestamp of when the data has imputed.

- **Glucose**:
  - *id* - Int → PRIMARY KEY AUTO_INCREMENT
  - *glucose* - Double → glucose prediction value
  - *userId* - Int → userId foreign key
  - *timestamp* - varchar(100) → contains the timestamp of when the data has imputed.